Software Requirements Specification

Incarus

Version 1.0

Prepared by Kyler M^{c}Lees, Roman Bureacov, and Riley

Hopper Turquoise

Tortoise

07/14/2025

Table of Contents

Revision History

Name	Date	Reason For Changes	Version
Kyler M^{c}Lees	7/14/2025	Populate general information	1.0
Riley Hopper	8/22/2025	Updated for product shipment	1.0

1. INTRODUCTION

1.1 Purpose

The purpose of this SRS document is to describe the requirements and details of a software system that will simulate a real-time Dungeon Crawler adventure. It will explain the inner workings and the goals of the system in order to deliver an enjoyable experience for users. It will explain system constraints, interfaces, and interactions with outside applications such as databases.

1.2 Intended Audience and Reading Suggestions

This document is intended for the application developers looking to review and evaluate the applications development process, and their own efforts on its development. It is a reference document for them to build a version 1 that is operable by the end-user. The document is also intended for the head stakeholder, Tom Capaul.

1.3 Project Scope

The software system in question will be a real-time Dungeon Crawler game. It will be designed to allow players to set difficulty levels, save and load progress from previous sessions, maneuver through “dungeons” that may contain collectable objects, obstacles, or enemies, and complete the objective provided by the system. This system will allow the players to tailor an entertaining and or challenging experience to their liking.

Players will be allowed to pick a character with exclusive skills. They will take these characters from room to room in a dungeon that is randomly generated. The player will be assumed to use a keyboard in order to move their character through dungeons. The gameplay will be real time, but it will allow the player to pause the game to freeze it in place until the game is resumed. There will be enemies populating some rooms in the dungeon that will attempt to prevent the player from completing their objective. Successfully doing so will result in the player losing the game and restarting from the beginning of a new dungeon. The player will be looking for the four pillars of OO. If the player can find all pillars, then they will have completed their objective and will win the game. Any time before the player wins or loses, they will be able to use the GUI to save their current progress, position, and stats. At any time, they player will have the ability to use the GUI to load a save file to resume from saved progress, positions, and stats.

The software needed is going to be the files of the game, as well as the libraries of SDL3 that allow the game GUI to open. The hardware required will be a device capable of downloading or copying the files into storage, accessing the files, and running an executable. A database will be needed to store data on the player and enemies. A keyboard and mouse are needed to interact with the dungeon as a character. This all culminates together to create an enjoyable experience for the players that is as fun as it will be challenges.

1.4 References

None.

2. Overall Description

2.1 Product Perspective

This gaming application originated as a new system where team members collaborated to identify project scope and design requirements. The game itself is designed for Computer platforms, and the student audience. It is built using C++ as production language and it uses SDL3 to handle platform dependent Windowing, Audio, and User interaction. SQLite is used for data handling; the application operates as a standalone system without network interactions. The system is expected to allow the user to play a bugless game that will go through multiple releases for patches and be available for Windows or Linux devices. The system will use assets including images and music created by the developers to improve immersion.

2.2 Product Features

The major features of the game will be as follows:

- Navigating a Menu to select a difficulty and start a game.
- Save and Load functions to store or pull data to and from a database.
- Combating enemies in a Real-time combat system allows a player to have a challenging

experience that may cause them to lose.
- A dungeon will be randomly generated every time, making each experience unique, and therefore engaging.

2.3 Design and Implementation Constraints

The operating system constraints will consist of the minimums:
- System RAM: 8 GB
- Free storage: 10 GB
- Storage type: SSD or HDD
- CPU: x64
- Graphical Interface: yes
- Operating Systems: Windows XP or later, Linux with a graphical interface

2.4 Documentation to be Used

We will make use of the following documentation, in addition to AI systems and search engines:
- SDL3 wiki: https://wiki.libsdl.org/SDL3/FrontPage
- cppreference: https://en.cppreference.com/w/
- MSVC compiler options: https://learn.microsoft.com/en-us/cpp/build/reference/compiler options?view=msvc-170

2.5 Dependencies

Our game will be developed using C++, specified as follows:
- Language: C++, CMake
- C++ standard: C++23
- Cmake version minimum: 3.20
- Compiler: MSVC
- Compiler flags: c++latest
- IDE: Clion

We will depend on the libraries:
- SDL3, release 3.2.18, x64
- SDL3_Image.
- SDL3_ttf.
- OpenGL.
- SQLite 3.50.4 .
- Google Tests 1.17.0.

3. System Features

3.1 Moving the Playable Character

3.1.1 Description and Priority

High priority, the user should be able to move the character at their whim however they want (within the bounds in the playable area(s)).

3.1.2 Stimulus/Response Sequences

1. The user presses a movement key on their keyboard
2. The engine takes in the action
3. The engine updates the position of the playable character
4. The view updates the visual location of the playable character

3.1.3 Functional Requirements

REQ-1: Game engine to update the player states
REQ-2: Hooks to the user keyboard
REQ-3: Keybinds defined

3.2 Combat

3.2.1 Description and Priority

High priority the user should be able to use the action button to attack, at any time. The System should react accordingly.

3.2.2 Stimulus/Response Sequences

1. The user uses the combat action button.
2. The program will enqueue the attack event into the process queue
3. The clock ticks and the engine processes the attack event
4. The engine gets the character's weapon and projects an attack hitbox
5. The engine then checks what entities' hitboxes intersect with the action's hitbox.
6. The engine updates any entities that intersect with the attack hitbox.
7. The entities react accordingly.

3.2.3 Functional Requirements

REQ-1: When the user presses the combat action button, an attack is performed. REQ-2: The correct animation plays with an attack is performed.
REQ-3: The engine corrected flags any entites within the attack area.
REQ-4: The engine updates any entities that overlap with the attack.

3.3 Real-Time

3.3.1 Description and Priority

High priority. The staple of the game. This will define how the player interacts with the environments and enemies. This will allow the game to progress in real-time, where the

entities and the player interact on their own accord rather than in sequence of one-another, being able to act and interact concurrently.

3.3.2 Stimulus/Response Sequences

1. The game engine ticks every fixed interval of time
2. On every tick, the game engine will read inputs of the player and entities
3. The game engine will then proceed to update the state of entities based on actions and interactions.

3.3.3 Functional Requirements

REQ-1: A game engine that will handle updating entities
REQ-2: A clock that ticks for fixed intervals of time

3.4 Potions

3.4.1 Description and Priority

Medium. Potions that heal the player when they are low on health

3.4.2 Stimulus/Response Sequences

1. Player finds a potion.
2. Player presses the heal button to heal.
3. The players health is healed up to its maximum

3.4.3 Functional Requirements

REQ-1: Potions to be spawned in the dungeon.
REQ-2: A system for storing the number of potions.

3.5 Animations

3.5.1 Description and Priority

Medium. Entities should be animated using fixed sequences of static images.

3.5.2 Stimulus/Response Sequences

1. Entity performs an action
2. The game engine detects the action
3. The game engine will start an animation sequence

3.5.3 Functional Requirements

REQ-1: Game engine to detect the action
REQ-2: Image sequences

3.6 Random Dungeon Generation

3.6.1 Description and Priority

High. This is a core component of the dungeon crawler. The game will create a random dungoen every time. This will randomly generate where the rooms are and will place the objectives in rooms accordingly.

3.6.2 Stimulus/Response Sequences

1. User starts the game
2. The game will generate a map with rooms and their corresponding items (if applicable)
3. The game will load the player in a starting room of the maze

3.6.3 Functional Requirements

REQ-1: Maze generation algorithm

3.7 Dodge Roll

3.7.1 Description and Priority

This action will allow the user to dodge the incoming attacks from enemies, while dodging the user will take no damage. They also move in the direction the player is facing. This is a low priority.

3.7.2 Stimulus/Response Sequences

1. The user presses the dodge button.
2. The engine takes action.
3. The engine updates the position of the playable character.
4. The view updates the visual location of the playable character.
5. The engine stops the player from taking damage.

3.7.3 Functional Requirements

REQ-1: When the dodge button is pressed, the correct animation is played.
REQ-2: When the dodge button is pressed, the player moves in the correct direction.
REQ-3: When the dodge button is pressed, the player takes no damage.

4. External Interface Requirements

4.1 User Interfaces

The style guidelines for the user interface will follow a high fantasy aesthetic, with pixel art characters. The main game will have an interface that includes:

- Mini-Map.
- Health bar.
- Menu button.

4.2 Hardware Interfaces

The user will be able to use their Keyboard and Mouse to interact with the System. The systems speakers and monitor will be needed as well to show graphics and audio.

4.3 Software Interfaces

The game is connected to a SQLite database that will contain game data.

The game will also make use of SDL for graphics and audio.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The user must be able to run the game without, without lag.

5.2 Safety Requirements

The user may lose years of their life due to addictive gameplay, and mega epic mechanics.

5.3 Security Requirements

The product may cause social security and associated names to be leaked, but this should be a very minimal issue.

5.4 Software Quality Attributes

The game will be so usable you will forget how to walk.

6. Other Requirements

Trademark and patent the game so NO-ONE can remake it. Apply a DRM to it so it is impossible to pirate.

Appendix A: Glossary

Term	Definition
Players	End-users of the system
Dungeon	A collection of rooms strung together
Dungeon Crawler	A type of game where an end-user runs through a dungeon to complete an objective
Character	The entity an end-user will have direct control over
Enemy	An entity in the system that will try to prevent the end-user from completing an objective
Objective	The goal the end-user must complete to win the game
GUI	The interface the user will interact with to save game, load game, see and control the character through rooms and against enemies, and start a game at a designated difficulty
Entity	Game objects, such as enemies, the user character, and miscellaneous interactable objects.

Appendix B: Issues List

1. *The game doesn't run at 30fps.*
2. *The roll mechanic isn't implemented.*
3. *Animations don't work.*
4. *Hitbox collision is messy.*
5. *Ui is unimplemented.*
6. *There is no main menu.*